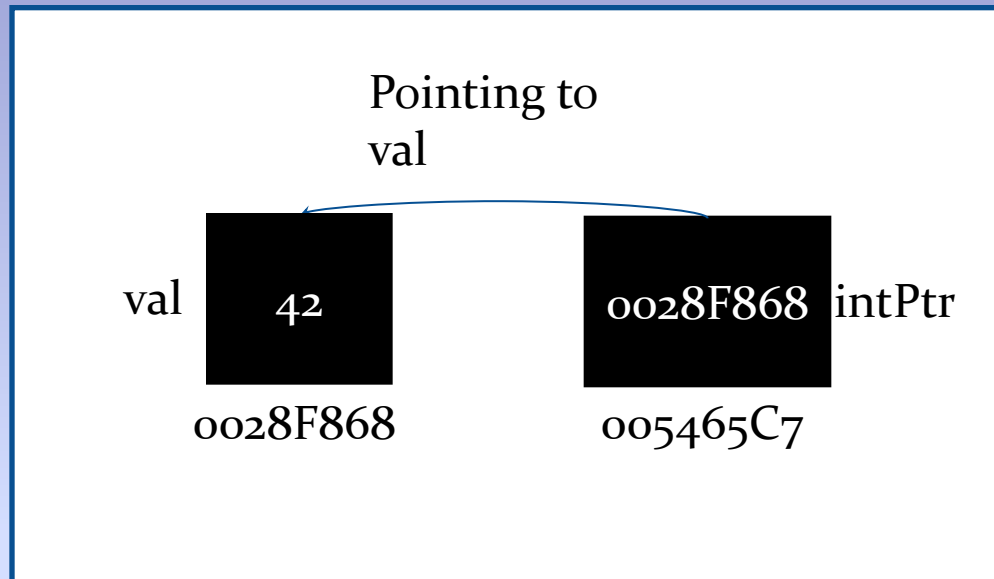


POINTERS

Pointers



Pointers

- Declaration of pointer to an int

```
int* intPtr;
```

- Initializing a pointer

```
int val = 42;  
int* intPtr = &val;
```

- The ampersand (or '&') before val means "take the address of val".
- The pointer intPtr is assigned the address of val, which is another way of saying that intPtr now points to val.

Pointers

- If we print out the value of `intPtr`, we'll get the address of `val`.
- To get the data that `intPtr` actually points at, we have to *dereference* `intPtr` with the *unary star operator* `*`

```
int val = 42;  
int *intPtr = &val;  
cout << "&val: " << &val << endl; //address of val  
cout << "intPtr: " << intPtr << endl; // ..again, address  
cout << "*intPtr: " << *intPtr << endl; // displays 42
```

- Note that the star (or `'*`') operator has many uses. When used in a binary context, it can mean multiplication. When used in a variable declaration, it means "this variable is a pointer". When used in an expression as a single-argument operator, it can mean "dereference". With so many uses, it's easy to get confused.

Pointers

- Since intPtr points to val, any changes that we make to val will also show up when we dereference intPtr:

```
int val = 42;
int* intPtr = &val;
cout << "val: " << val << endl; // displays 42
cout<<"*intPtr: "<<*intPtr<<endl;//displays 42
val = 999;
cout << "val: " << val << endl;// displays 999
cout << "*intPtr: " << *intPtr << endl; //
displays 999
```

Pointers

- You can declare a pointer to any type, not just int:

```
char ch= 'H';
```

```
char* chPtr = &ch;
```

```
cout << "ch: " << ch<< endl;
```

```
cout << "chPtr: " << chPtr << endl;
```

```
cout << "*chPtr: " << *chPtr << endl;
```

```
*chPtr = "!";
```

```
cout << "*chPtr: " << *chPtr << endl;
```

```
cout << "ch: " << ch<< endl;
```

Pointers

```
float val = 1.43;  
float* fltPtr = &val;  
cout << "val: " << val << endl;      // 1.43  
cout << "val: " << fltPtr << endl;    // address  
cout << "*fltPtr: " << *fltPtr << endl; // 1.43  
*fltPtr = 3.1416;  
cout << "*fltPtr: " << *fltPtr << endl; //3.1416  
cout << "val: " << val << endl;      // 3.1416
```

Pointers and **const**

- Pointers can be declared **const** in three ways:
 - The pointer itself can be declared **const**
 - The data the pointer points (“the pointee”) can be declared **const**
 - Both the pointer and the pointee can be declared **const**

Pointers and const

- **const** pointer, which means that once initialized it can't point at something else.

```
int val1 = 42;  
int * const intPtr = &val1;  
*intPtr = -1;           // okay  
int val2 = 999;  
intPtr = &val2;         // error!
```

Pointers and const

- **const** data, which means that the data that the pointer points to can't be changed

```
int val1 = 42;
```

```
const int * intPtr = &val1;
```

```
*intPtr = -1;           // error!
```

```
int val2 = 999;
```

```
intPtr = &val2;         // okay
```

Pointers and const

- Both of the above -- you can change neither the pointer nor the data it points to:

```
int val1 = 42;  
const int * const intPtr = &val1;  
*intPtr = -1;    // error!  
int val2 = 999;  
intPtr = &val2;  // error!
```

Pointers and const

- The variety of ways that you can use const can be confusing. One useful way to understand pointer declarations is to read them starting from the variable name reading from right to left .
 - `int * intPtr;` *//the variable intPtr is a pointer to an int.*
 - `const int * intPtr;` *//the variable intPtr is a pointer to an int //that is constant. Since the int is constant, we can't change it. We //can change intPtr, however.*
 - `int * const intPtr;` *//the variable intPtr is a constant pointer //to an int. Since intPtr is constant, we can't change it. However, we //can change the int.*
 - `const int * const intPtr;` *//the variable intPtr is a constant pointer to // an int that is constant. Everything is constant.*

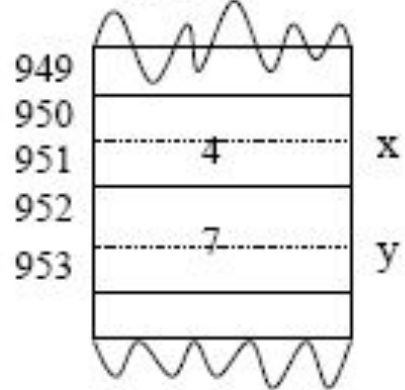
Example

```
int main ( )
{ int x=4, y=7;
  int *px, *py;
  px = &x; //int *px=&x;
  py = &y; //int *py=&y;
  cout << "x = " << x<<"y ="<<y;
  *px = *px + 10;
  *py = *py + 10;
  cout << "x = " << x<<"y ="<<y;
  return 0;
}
```

```
int x=4,y=7
```

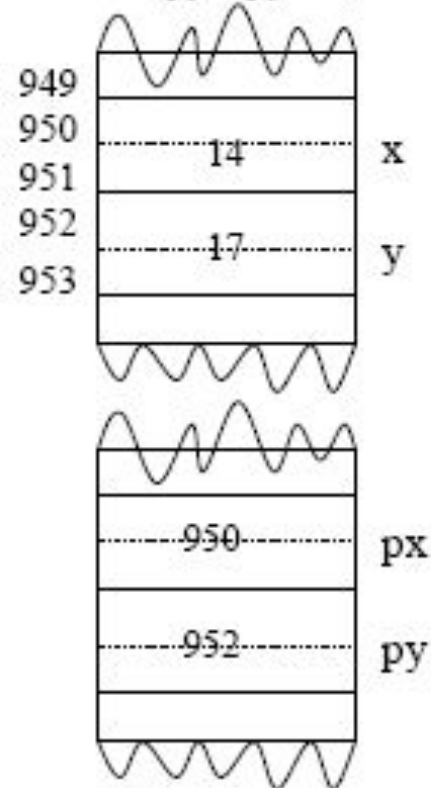
```
px=&x;
```

```
py=&y;
```



```
*px=*px+10;
```

```
*py=*py+10;
```



Operation of the addition using pointers

Discussion

```
int main (void)
{ int a;
  int *ptr1;
  char c;
  char *ptr2;
  int fred;
  ptr2=&c;      //ok. Char pointer init with add of a char
  ptr1=&fred;    //ok. int pointer init with the address of an int

  ptr1=&a ;      //ok. int pointer init with the address of an
                int
```

Discussion

```
ptr2=&fred;    // NO. cannot assign the address of an int to a char
ptr2=&'#';     // NO. cannot take the address of an implicit constant
ptr2=&25;      // NO. cannot take the address of an implicit
               // constant
ptr2=&(a + 3);  // NO. cannot take the address of an expression
}
```


Discussion

```
int main()  
{ int c,a=10;  
  int *p1=&a;  
  c=*p1; //equivalent expression "c=a"  
  *p1=*p1**p1; //equivalent expression "a=a*a"  
  (*p1)++; //equivalent expression "a++"  
  c=*&a; //equivalent expression "c=a"  
}
```

Discussion

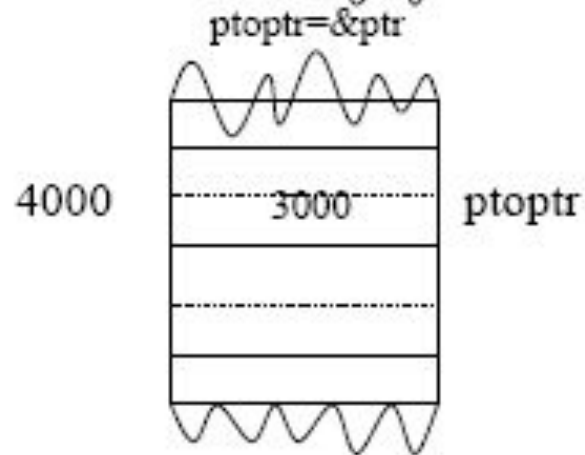
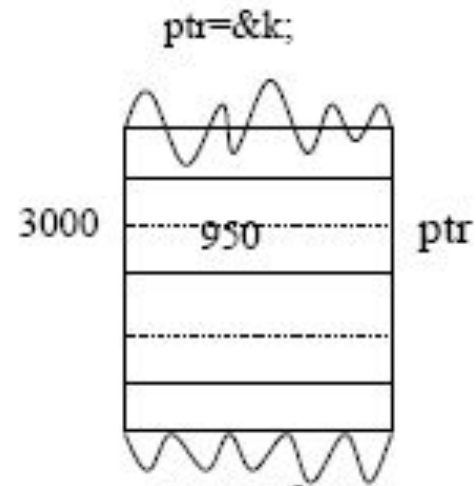
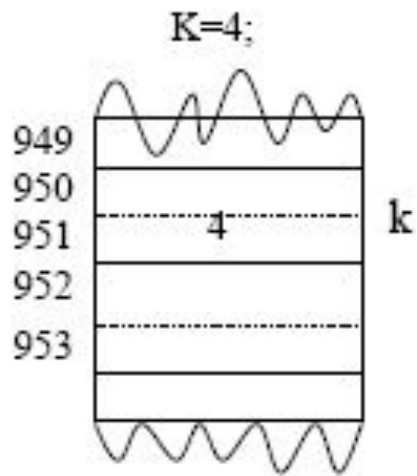
```
int a = 3;  
int b = 4;  
int *pointerToA = &a;  
int *pointerToB = &b;  
int *p = pointerToA;  
p = pointerToB;  
cout<< a << b << *p;    // Prints 344
```

Pointer to Pointer

- `int k=4, *ptr, **ptoptr;`
- `ptr=&k`
- `ptoptr=&ptr;`

USE

- Creation of dynamically sizeable arrays
- Use of *new* and *delete*



- `int k=4,*ptr,**ptoptr;`
- `ptr=&k`
- `ptoptr=&ptr;`

```
int main ()
{
int x = 1;
int *px = &x;
int **ppx = &px;

cout << "x = " << x << endl;

*px = x+10;
cout << "x = " << x << endl;

**ppx = *px + x;
cout << "x = " << x << endl;

return 0;
}
```

```
int main ()  
{  
int x = 1;  
int *px = &x;  
int **ppx = &px;
```

```
cout
```

```
*px
```

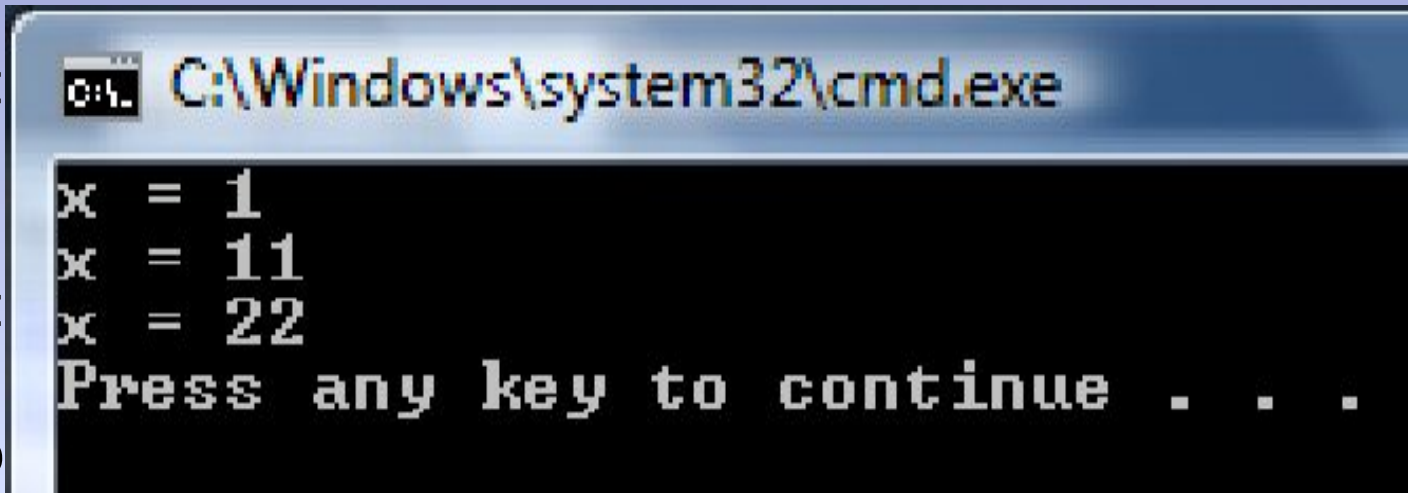
```
cout
```

```
**pp
```

```
cout << "x = " << x << endl;
```

```
return 0;
```

```
}
```



```
C:\Windows\system32\cmd.exe  
x = 1  
x = 11  
x = 22  
Press any key to continue . . .
```

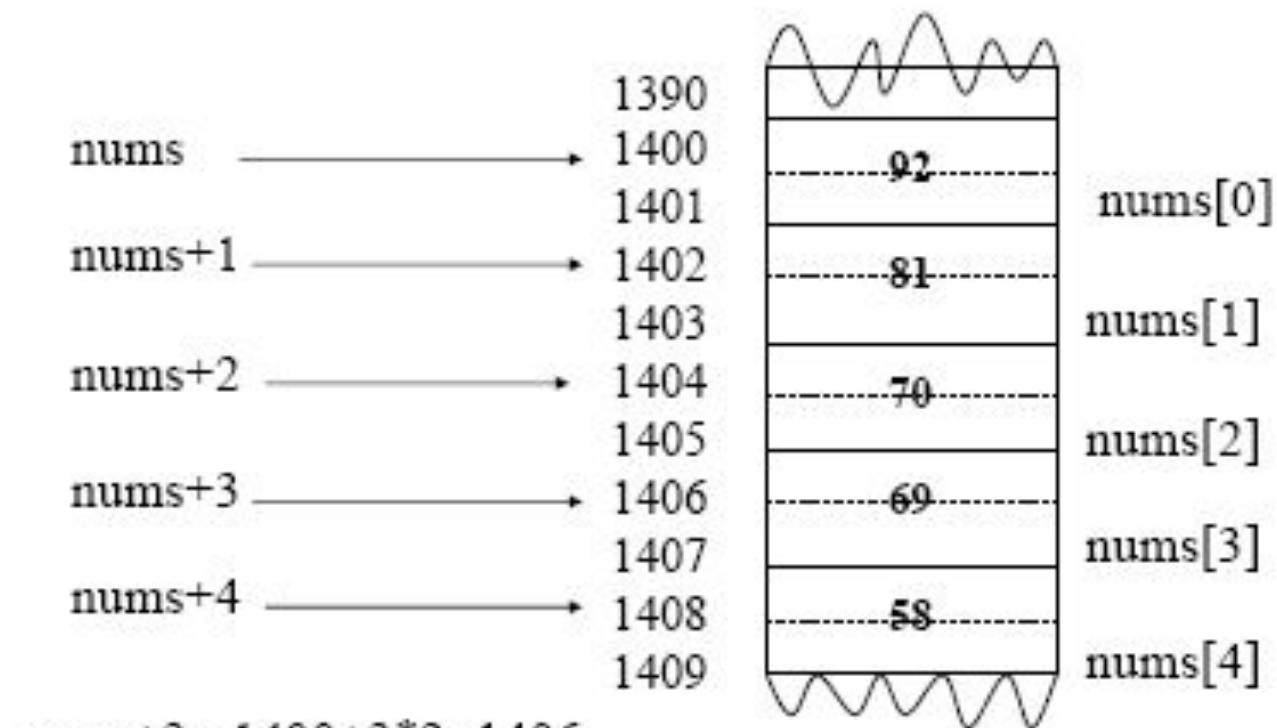
Pointers and Arrays

- In c++, you can treat pointers and arrays very similarly:

```
int array[ ] = {3,1,4,1,5,9};
int* arrayPtr = array;
cout<<"array[0]: " <<array[0] << endl;//displays 3
cout<<"arrayPtr[0]: "<<arrayPtr[0]<<endl;//displays 3
arrayPtr++;
cout<<"arrayPtr[0]: "<<arrayPtr[0]<<endl; // displays 1
array++; // error: arrays are const
```

Pointers and Arrays

- Arrays and pointers may be used interchangeably, but you can change what pointers point at, whereas arrays will always "point" to the same thing.



$$\text{nums}+3 = 1400 + 3 * 2 = 1406$$

number of bytes
per integer.

Scalar value